Eliminarea partilor nevizibile ale scenelor 3D din imagini - 1

Prof. univ. dr. ing. Florica Moldoveanu

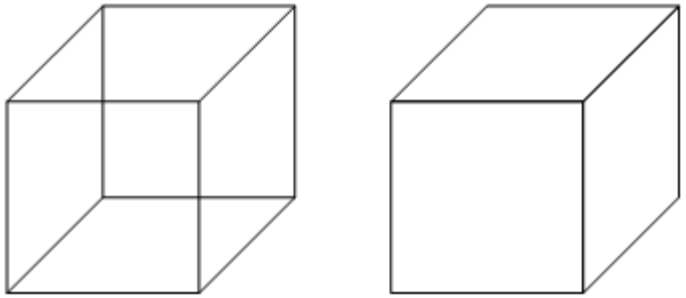
Eliminarea partilor nevizibile ale scenelor 3D, din imagini (1)

Vizibilitatea obiectelor dintr-o scena 3D depinde de:

– Pozitia observatorului si directia in care priveste

– Pozitionarea obiectelor din scena unul fata de celalalt

– Volumul vizual



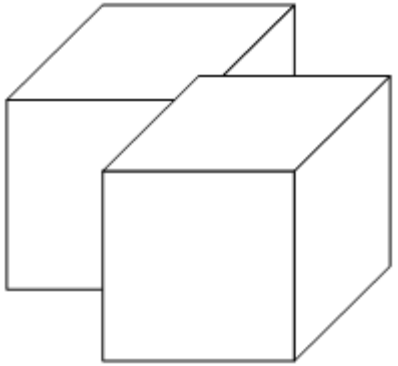
Partile unei scene 3D nevizibile într-o imagine sunt:

– Părți aflate total sau parțial în afara volumului vizual

– Fețe auto-obturate

– Părți obturate (ascunse) de obiecte ale scenei,

aflate în fața lor în raport cu observatorul



Eliminarea partilor nevizibile ale scenelor 3D, din imagini (2)

Efectuata la nivel de:

1. Obiecte sau grupuri de obiecte - **Object culling**:

   - **Frustum Culling**: eliminarea din banda grafica a obiectelor sau grupurilor de obiecte care sunt complet in afara volumului vizual
   - **Occlusion Culling**: – eliminarea din banda grafica a obiectelor sau grupurilor de obiecte care sunt complet obturate de alte obiecte

   - nu este efectuata de OpenGL/GPU
   - poate fi efectuata prin algoritmi implementati intr-o biblioteca folosita de aplicatia grafica 3D (motorul grafic)

2. Fețe ale obiectelor - ***Back face culling:*** eliminarea din banda grafica a fețelor auto-obturate ale obiectelor (este efectuata pe GPU)**.**

Eliminarea partilor nevizibile ale scenelor 3D, din imagini (3)

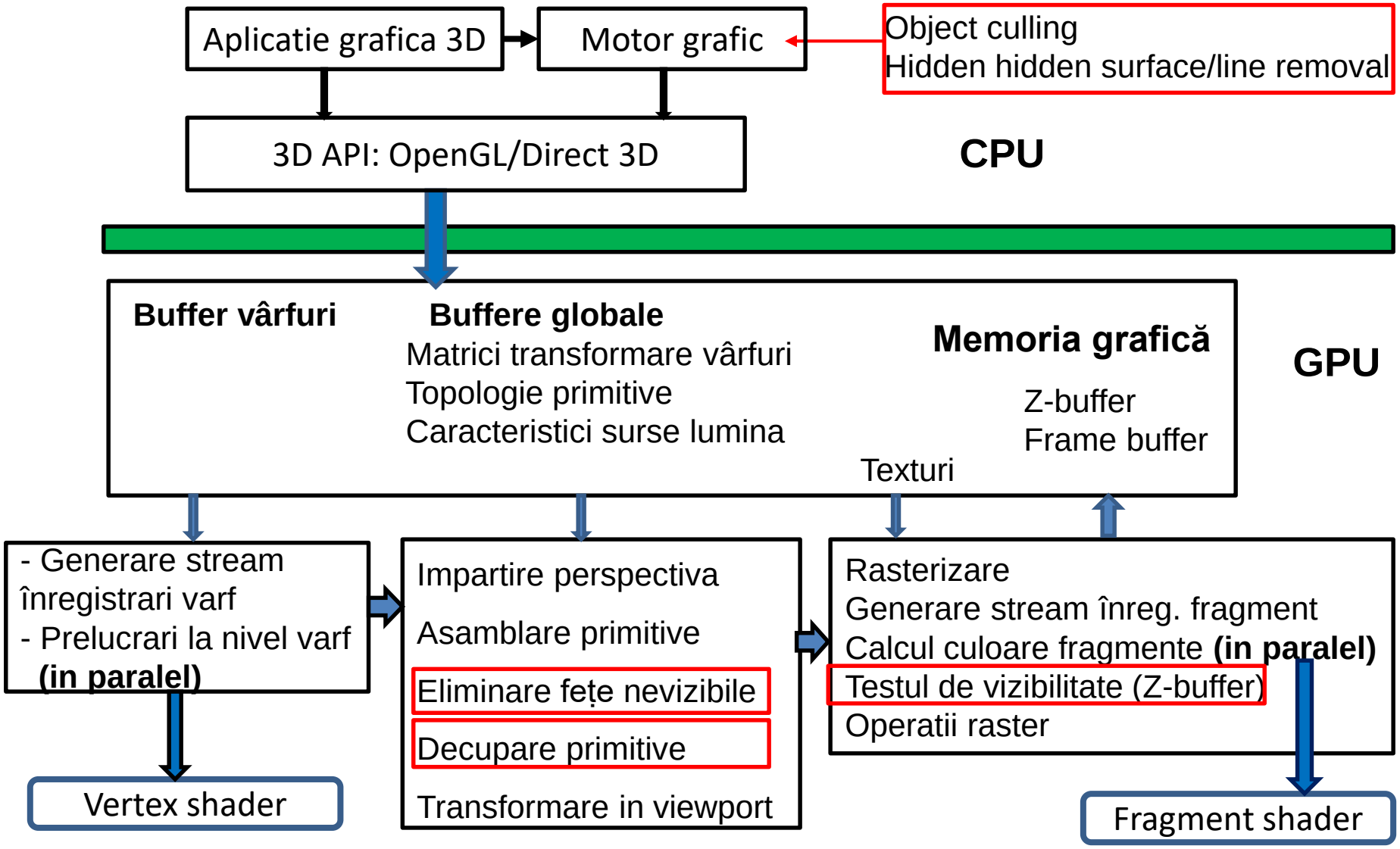
3. Primitive grafice (poligoane, linii):

- Eliminarea primitivelor aflate total sau partial in afara volumului vizual prin operatia de **decupare (clipping)** efectuata de GPU

- Eliminarea partilor obturate (ascunse) ale primitivelor grafice: ***hidden surface/ hidden line removal***

  - poate fi efectuata prin algoritmi implementati intr-o biblioteca folosita de aplicatia grafica 3D (motorul grafic): BSP, Pictorului, ş.a.
  - este efectuata de GPU, ca operatie raster la nivel de fragment (algoritmul Z-buffer)

Eliminarea partilor nevizibile in banda grafica



Inregistrare varf: intrare Vertex shader

Inregistrare fragment: intrare Fragment shader

Eliminarea fetelor auto-obturate pentru poliedre convexe (Back face culling)(1)

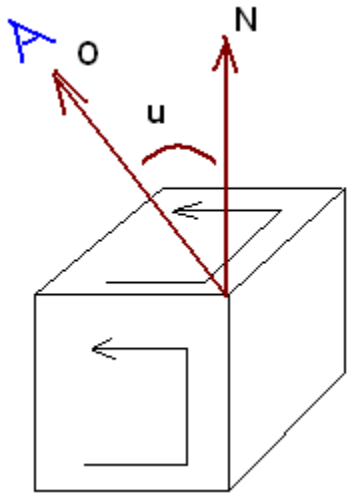
- Poate reduce substantial numarul de poligoane rasterizate.

Algoritmul:

- Observatorul si obiectul sunt raportati la acelasi sistem de coordonate.

- Conturul fiecarei fețe a obiectului este orientat in sens trigonometric atunci cand poliedrul este vazut din exterior.

- Observatorul este situat in afara obiectului.



Fața este vizibila daca : $0 <= u < 90^o$ sau $0 < \cos(u) <= 1$

N: normala la față

O: vectorul orientat catre observator

Produsul scalar: $N \bullet O = |N| * |O| * \cos(u) \rightarrow \cos(u) = N \bullet O / (|N| * |O|) = Nu \bullet Ou$

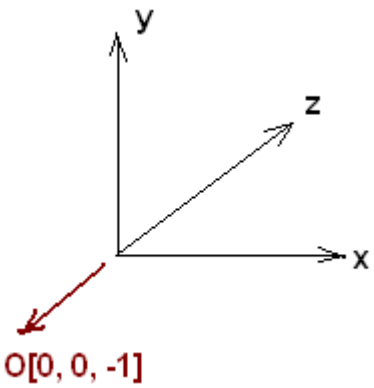
Conditia de vizibilitate: Nu • Ou > 0 (produsul scalar al versorilor)

Determinarea fetelor auto-obturate pentru poliedre convexe(2)

Determinarea feţelor auto-obturate poate fi efectuata:

1. In sistemul coordonatelor globale (in care este definit si observatorul – camera virtuala)
2. In sistemul de coordonate observator
3. In sistemul coordonatelor de decupare (dupa aplicarea transformarii de proiectie)

In cazul 3



Sistemul coordonatelor de decupare

Nu • Ou = [nx, ny, nz] • [0, 0, -1] = - nz

Conditia de vizibilitate devine: nz <0

Dupa transformarea de proiectie pozitia observatorului este la infinit, pe axa z negativa.

Vectorul orientat catre observator este acelasi pentru orice punct al unui obiect

Determinarea fetelor auto-obturate pentru poliedre convexe(3)

Calculul normalei la o faţă a poliedrului

Fie V1(x1, y1, z1), V2(x2, y2, z2), V3(x3, y3, z3), 3 varfuri succesive ale conturului feţei.
Normala la faţă se poate obtine calculand produsul vectorial: (V1-V2)x(V2-V3)

$$\mathbf{N} = (V1-V2)\times(V2-V3) = \begin{vmatrix} i & j & k \\ x2-x1 & y2-y1 & z2-z1 \\ x3-x2 & y3-y2 & z3-z2 \end{vmatrix}$$

$$= [(z3-z2)*(y2-y1) - (z2-z1)*(y3-y2)]*\mathbf{i} - [(x2-x1)*(z3-z2) - (x3-x2)*(z2-z1)]*\mathbf{j} + [(x2-x1)*(y3-y2) - (x3-x2)*(y2-y1)]*\mathbf{k}$$

unde **i**, **j**, **k** sunt versorii directiilor axelor sistemului de coordonate carteziene 3D

Normalele:

- Pot fi calculate in sistemul coordonatelor obiect si atasate varfurilor in aplicatia OpenGL → sunt transformate in sistemul coordonatelor de decupare, in vertex shader.
- Nu sunt atasate varfurilor: sunt calculate pe GPU dupa asamblarea primitivelor, folosind varfurile transformate in sistemul coordonatelor de decupare.

Determinarea fetelor auto-obturate pentru poliedre convexe(4)

Pentru eliminarea feţelor auto-obturate se apeleaza urmatoarele funcţii OpenGL:

glCullFace(GL_FRONT/ GL_BACK/GL_FRONT_AND_BACK);

- eliminare fete: din faţă / din spate / toate (nu vor fi afisate fețe, ci numai alte primitive: puncte, linii)

glFrontFace(GL_CCW/GL_CW);

- specifica orientarea feţelor din faţă: trigonometrica/sensul acelor de ceas

➢ Daca ciclul de varfuri al fiecarei fete este orientat trigonometric atunci cand obiectul este privit din exteriorul său:

- atunci cand observatorul este in spatiul exterior obiectului se va apela glFrontFace(GL_CCW)
- atunci cand observatorul este in spatiul interior obiectului se va apela glFrontFace(GL_CW)

(vazute din interior, feţele obiectului au conturul orientat anti-trigonometric).

Algoritmul z-buffer(1)

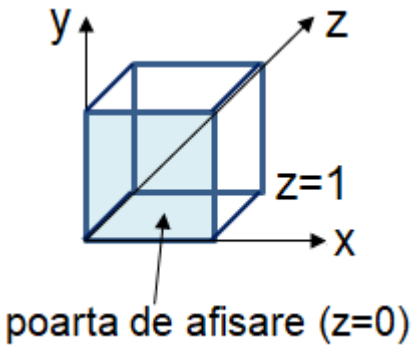
- Algoritm de eliminare a fragmentelor nevizibile ale primitivelor grafice.

- Executat de GPU, fiind integrat in procesul de rasterizare a primitivelor

- Observatorul este la infinit, pe axa z negativa

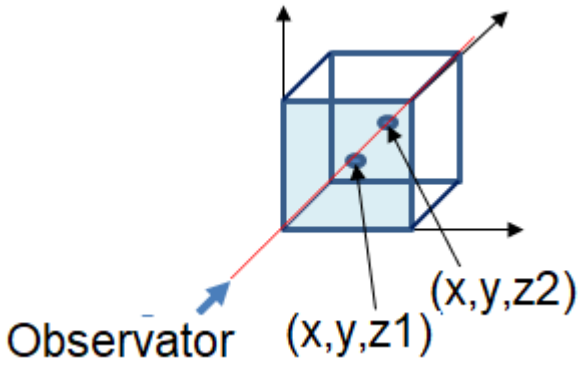
- Asupra primitivelor se efectueaza proiectie ortografica

  in planul XOY (poarta de afisare)



Daca doua fragmente aflate pe acelasi proiector, rezultate din rasterizarea a doua primitive diferite, au coordonata z diferita:

- Fragmentul avand coordonata z mai mica va fi afisat în pixelul (x,y).



Algoritmul z-bufer(2)

- Primitivele grafice (triunghiuri, linii) in care a fost descompusa scena 3D sunt rasterizate in ordinea in care au fost transmise din programul de aplicatie (nu sunt sortate).

- Buffer-ul imagine (frame buffer) este actualizat pe masura rasterizarii primitivelor, astfel:

daca (fragmentul curent se proiecteaza in pixelul (x,y) si
z-fragment < z-fragment afisat in pixelul (x,y))
atunci
actualizeaza culoarea pixelului (x,y) in bufer-ul imagine, la culoarea fragmentului curent

Rezulta: este necesar sa se memoreze coordonatele z ale fragmentelor afisate in pixelii imaginii



Z-buffer: - tablou bidimensional cu numar de elemente egal cu numarul de pixeli ai suprafetei de afisare
- în memoria grafica

Algoritmul z-bufer(3)

Algoritmul z-buffer (integrat in procesul de rasterizare):

*Initializeaza buffer-ul imagine la culoarea de fond
*Initializeaza Z-buffer la coordonata z maxima (z=1)

Pentru fiecare fragment f(x,y,z) rezultat din rasterizarea unei primitive

* calculeaza culoarea fragmentului (in fragment shader)

//testul de vizibilitate (adancime)

daca z< Z-buffer[y][x] atunci

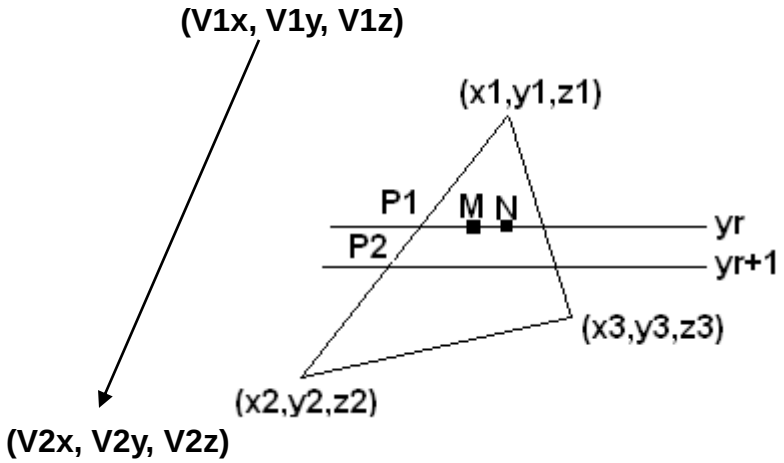
{ Z-buffer[y][x] = z

*actualizeaza culoarea pixelului (x,y) in buffer-ul imagine folosind culoarea fragmentului f

}

Algoritmul z-bufer(4)

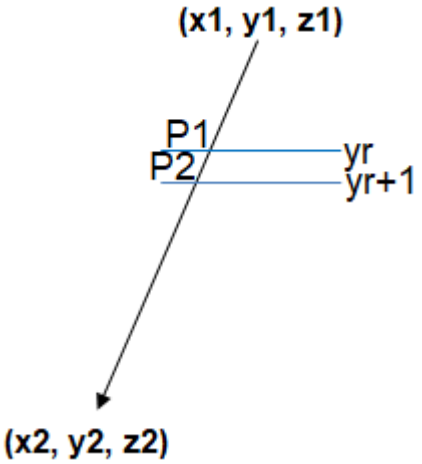
Calculul coordonatei z a unui fragment



Coordonatele z ale fragmentelor rezultate la rasterizarea vectorilor si a suprafetelor triunghiulare se obtin prin calcul incremental, in algoritmul de rasterizare.

Algoritmul z-bufer(5)

Calculul coordonatei z la rasterizarea vectorilor



x = x1 + t(x2-x1)
y = y1 + t(y2-y1) Ecuatiile parametrice ale vectorului
z = z1 + t(z2-z1)

yr, yr+1: ordonatele a 2 linii raster (ecran) care intersecteaza vectorul

y(P1) = yr = y1 + t'(y2-y1) → t' = (yr-y1)/(y2-y1)

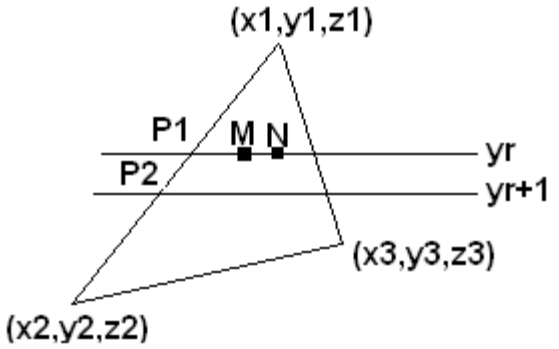
z(P1) = z1 + (yr-y1)(z2-z1)/(y2-y1) = z1 + (yr-y1)*K,
K este o constanta la rasterizarea vectorului

z(P2) = z1 + (yr+1-y1)*K = z1 + (yr-y1)*K + K

Z(P2) = z(P1) + K → coordonatele z ale fragmentelor de pe vector se obtin prin calcul incremental.

Algoritmul z-bufer(5)

Calculul coordonatei z la rasterizarea poligoanelor



P1(xP1, yr, zP1), P2(xP2, yr+1, zP2)

m- panta laturii (x1,y1,z1) – (x2,y2,z2)

xP2 = xP1 + 1/m (vezi curs - rasterizare)

A*x + B*y + C*z + D = 0 ec planului poligonului

zP1 = (-A*xP1 - B*yr –D)/C

zP2 = (-A*xP2 - B*(yr +1) – D)/C = (-A(xP1 + 1/m) - B*yr –B – D)/C = zP1 + (-A/m –B) → **zP2 = zP1 + K1**

M(xM, yr, zM), N(xM + 1, yr, zN)

zN = (-A*(xM + 1) – B*yr – D)/ C = zM + (-A/C) → **zN = zM + k2**

Observatie: atunci cand un vector se suprapune pe suprafata unui poligon pot aparea defecte datorita modului diferit in care se calculeaza coordonatele z ale fragmentelor care se afiseaza in aceiasi pixeli!

Algoritmul z-bufer(6)

Aprecieri asupra algoritmului z-Buffer

1) Numarul de comparatii de valori z pentru un pixel = numarul de fragmente care se proiecteaza in acel pixel.

   Timpul de calcul tinde sa devina independent de numarul de poligoane: in medie, numarul de pixeli acoperiti de un poligon este invers proportional cu numarul de poligoane.

2) "Depth complexity": numarul de suprascrieri ale unui pixel in buffer-ul imagine, la generarea unui cadru imagine

   - ❖ calcul culoare fragment: model iluminare, utilizare texturi

3) Pentru o precizie buna, este necesara o reprezentare pe 32 biţi a valorilor din z-buffer.

Algoritmul BSP (Binary Space Partitioning)(1)

- Initial, arborele BSP a fost folosit numai pentru operatia de eliminare a obiectelor (grupurilor de primitive) nevizibile din banda grafica (object culling).
- Ulterior a devenit algoritm pentru reprezentarea si afisarea scenelor 3D cu "eliminarea" partilor nevizibile.
- Este eficient pentru eliminarea partilor nevizibile ale poligoanelor (hidden surface removal) dar poate fi folosit si pentru eliminarea partilor nevizibile ale liniilor dintr-o scena compusa din poligoane si linii.

Intrarea: lista primitivelor care compun scena 3D
(nu are importanta din care obiect face parte fiecare primitiva)

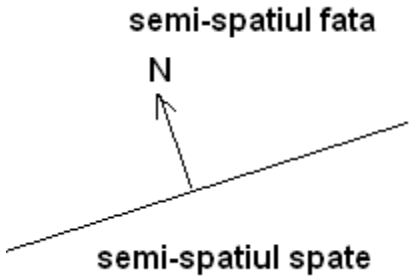
–Scena 3D este reprezentata printr-un arbore binar: arborele BSP

–Arborele BSP al scenei este independent de pozitia observatorului (reprezinta scena in sistemul coordonatelor globale).

Algoritmul BSP (2)

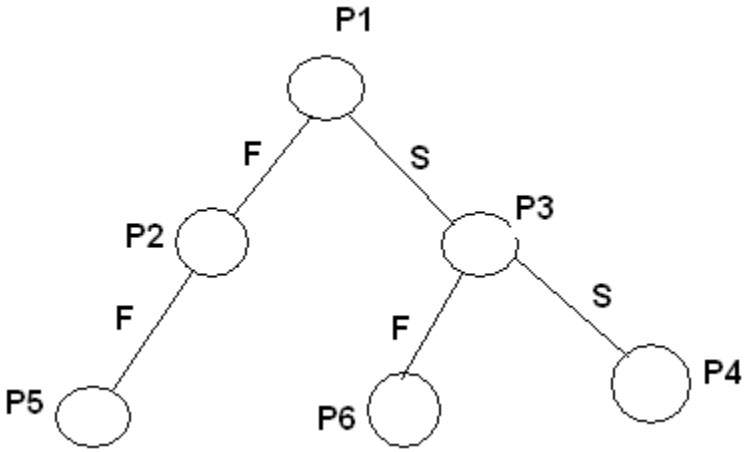
Construirea arborelui BSP al unei scene

▪Fiecare nod al arborelui corespunde unui plan de partitionare a spatiului 3D (de regula, planul unui poligon al scenei)



- Fiecare plan de partitionare imparte spatiul in 2 semi-spatii:
  - –cel din faţa planului (de aceeasi parte cu normala la plan)
  - –cel din spatele planului

1. Se incepe cu un poligon oarecare din lista (de regula primul), pentru care se creaza nodul radacina al arborelui



- Primitivele din semi-spatiul “faţă” formeaza “lista-faţă”, care va genera subarborele “faţă” al nodului
- Primitivele din semi-spatiul “spate” formeaza “lista-spate”, care va genera subarborele “spate” al nodului.

2. Se alege un poligon din lista-faţă si se creaza nodul radacina al subarborelui “faţă, etc.

Algoritmul BSP (3)

Algoritmul de creare a arborelui BSP:

arbore * creareBSP (Poligon * LP)

{ Poligon P, * ListaFata, * ListaSpate, * ListaNod;

daca (LP este vida) return NULL;

ListaFata = ListaSpate = ListaNod = NULL;

* alege un poligon P din LP;

* elimina P din LP; adauga P la ListaNod;

pentru (fiecare primitiva Q din LP) executa

{ **daca** (Q este in semispatiul față al planului lui P) **atunci**

* adauga Q in ListaFata

altfel

Algoritmul BSP (4)

daca (Q este in semispatiul spate al planului lui P) **atunci**

* adauga Q in ListaSpate

altfel

daca Q este in acelasi plan cu P **atunci**

* adauga Q in ListaNod

altfel // Q este intersectata de planul lui P

* divizeaza Q cu planul lui P

* adauga fiecare primitiva rezultata in ListaFata sau ListaSpate, in functie de pozitia sa

} // pentru fiecare primitiva

arbore * radacina = combina(creareBSP(ListaFata), creareBSP(ListaSpate));

*ataseaza ListaNod la radacina;

return radacina;

}

Algoritmul BSP (5)

Afişarea arborelui BSP al unei scene

-Tine cont de pozitia observatorului

- Afisare "back-to-front": se incepe cu primitiva cea mai indepartata de observator

- Se porneste din radacina arborelui si se avanseaza in arbore pana la frunze, in functie de pozitia observatorului fata de planul atasat fiecarui nod.

- In fiecare nod: se avanseaza in subarborele nodului care este opus observatorului fata de plan.

```
void afisareBSP(arbore * A, Pozitie Observator)
{ daca (! A) return;
  daca (Observator este in semispatiul fata al planului radacinii arborelui A) atunci
    { afisareBSP(A->spate); *afisare primitive din nodul radacina; afisareBSP(A->fata);}
  altfel
    { afisareBSP(A->fata); *afisare primitive din nodul radacina; afisareBSP(A->spate);}
}
```

Algoritmul BSP (6)

- **Eliminarea obiectelor aflate in afara volumului visual (Frustum culling):**

➢ Daca planul de partitionare al unui nod nu intersecteaza volumul vizual, atunci numai subarborele aflat de aceeaşi parte cu volumul vizual va fi afişat, celalalt subarbore fiind eliminat din banda grafica. La fel şi primitivele ataşate nodului (fiind în afara volumului vizual).

```
void afisareBSPcull (arbore * A, Pozitie Observator)
{ daca (! A) return;
  daca (volumul vizual este în fața planului radacinii arborelui A) atunci
        afisareBSPcull(A→fata); return;
  daca (volumul vizual este in spatele planului radacinii arborelui A) atunci
        afisareBSPcull(A→spate); return;
  //volumul vizual este intersectat de planul radacinii
    daca (Observator este in semispatiul fata al planului radacinii) atunci
     { afisareBSP(A->spate); *afisare primitive din nodul radacina; afisareBSP(A->fata);}
   altfel
    { afisareBSP(A->fata); *afisare primitive din nodul radacina; afisareBSP(A->spate);}
}
```

Algoritmul BSP (7)

Aprecieri

- Arborele nu trebuie sa fie construit pentru fiecare cadru imagine: avantaj pentru scenele statice
- Modificarea pozitiei observatorului → executia functiei de afisare (daca scena nu s-a modificat)
- Permite eliminarea din banda grafica a unui numar mare de poligoane care sunt in afara volumului vizual (Object culling)
- Pentru obiectele dinamice se pot face diverse adaptari ale algoritmului; de ex:

- pentru fiecare cadru imagine se insereaza in arborele BSP cu obiectele statice, noduri corespunzand obiectelor dinamice;

- se transmite arborele BSP static nemodificat si obiectele dinamice vizibile, eliminarea partilor obturate de acestea fiind realizata in algoritmul z-buffer (la rasterizare).

Algoritmul BSP (8)

Aprecieri (cont):

- Daca primitivele grafice sunt trimise in banda grafica in ordinea “din spate in faţă”, nu mai este necesar testul de adâncime la rasterizarea primitivelor (fragmentele care se proiecteaza in acelasi pixel sunt suprascrise).
- Testul de adancime efectuat asupra fragmentelor la rasterizare poate fi activat/dezactivat:

  glEnable(GL_DEPTH_TEST) / glDisable(GL_DEPTH_TEST)

- Daca se foloseste algoritmul BSP cu afisare «back-to-front» impreuna cu testul de adancime, numarul de suprascrieri ale pixelilor este mai mare decat in cazul unei ordini aleatoare a poligoanelor (algoritmul z-buffer fara BSP).
- Daca se foloseste BSP cu afisare «front-to-back» impreuna cu testul de adancime, afisarea poate fi mai rapida decat folosind BSP «back-to-front» fara testul de adancime, in functie de amplasarea poligoanelor in scena.
- Construirea si afisarea arborelui BSP sunt functii de management al scenei 3D, implementate de regula intr-un motor grafic 3D.

Algoritmul pictorului(1)

– Algoritm din categoria “Hidden surface removal”.

– Intrare: lista poligoanelor care alcatuiesc scena 3D, transformate in spatiul de afisare

– Ideea: eliminarea partilor nevizibile se rezolva prin afisare “din spate in fata”, la fel ca in pictura.

➢ Nu este necesar testul de adancime la rasterizarea poligoanelor.

Forma generala a algoritmului

1. Se calculeaza “extensia” fiecarui poligon din listă pe axele OX, OY, OZ (xmin,ymin,zmin – xmax, ymax, zmax): paralelipipedul incadrator al poligonului, cu feţele paralele cu planele principale ale sistemului de coordonate.

2. Se ordoneaza poligoanele crescator dupa coordonata zmin a fiecarui poligon: primul in lista va fi cel mai apropiat de observator.

3. Se descompun poligoanele ale caror extensii pe axa OZ se suprapun, astfel incat extensiile lor pe axa OZ sa fie disjuncte.

4. Se transmit in banda grafica poligoanele incepand cu ultimul din lista (cel mai indepartat de observator).

Algoritmul pictorului(2)

Pasul 3. Se descompun poligoanele ale caror extensii pe axa OZ se suprapun

- Sunt multe aplicatii in care acest pas nu este necesar, poligoanele fiind amplasate in plane de Z-constant: cartografie, generarea straturilor circuitelor imprimate, etc.
- Poate fi optimizat, stiind ca nu intotdeauna atunci cand extensiile pe axa OZ se suprapun este necesara descompunerea poligoanelor; descompunerea nu este necesara daca:

  - extensiile pe axa OX nu se suprapun

  - extensiile pe axa OY nu se suprapun

  - proiectiile poligoanelor nu se suprapun in imagine

- Testele se efectueaza progresiv, in functie de complexitatea calculelor presupuse.

- **Principalul efort de calcul:** sortarea listei de poligoane si descompunerea, daca este necesara.